# Beszúrás, összefésülés, sorbarendezések. Buborékrendezés, rendezés közvetlen kiválasztással, rendezés iterált beszúrással. Az utóbbi rendezés optimalitása. Legnagyobb és következő elemek megtalálása. Lépésszámok a fenti algoritmusokra.

2018. december 10., hétfő 2:28

## Beszúrás

Elkészítünk egy *n-1* elemű listát (pl. egy névsort). Ezután az *n*-edik elemet a helyére szeretnénk tenni. Ez úgy értendő, hogy a listában szereplő bármely másik dologgal összehasonlítva meg tudjuk mondani, hogy előtte vagy utána következik.
Ehhez a legegyszerűbb eljárás végigmenni a listán az első olyan tételig, amelyet már a beszúrandó nem előz meg, ekkor a szóban forgó *n*-ediket beszúrjuk elé.
Ez az eljárás néhány alkalommal lehet, hogy a legjobb, de nagyon sok összehasonlítást igényel. (legrosszabb esetben *n-1* darab összehasonlításra van szükségünk)

## Összefésülés

Adott két *n* hosszúságú lista. Egyesítenünk kell őket. Mindkét listának rendelkeznie kell egy jól meghatározott sorrenddel, hogy össze tudjuk fésülni őket. Ezt minél kevesebb összehasonlítással szeretnénk megoldani.
Tegyük fel, hogy a két lista $(x_1, \ldots, x_n)$ és $(y_1, \ldots, y_n)$. Először hasonlítsuk össze $y_1$ elemet rendre $x_1, x_2, \ldots$ elemekkel, míg meg nem találjuk az első olyat, melyet megelőz, ez mos legyen az $x_2$. Ezt tegyük be $y_1$ elé. Ezután hasonlítsuk össze $y_2$-t az $x_2, x_3, \ldots$ elemekkel, amíg nem találunk olyat, amelyet $y_2$ megelőz, ekkor ezt betesszük $y_2$ elé stb.
Ennek az eljárásnak a hátránya, hogy $x_1$ elemet sokat kell mozgatni. Ez plusz tár használatával kiküszöbölhető.
Ilyenkor összehasonlítjuk a két első elemet. Leírjuk azt, amelyik megelőzi a másikat egy külön helyre és töröljük az eredeti helyéről. Ezt addig ismételjük, amíg az egyik lista el nem fogy. Ezután a "maradék" lista elmeit beírjuk az új lista végére. Ezzel az eljárással minden elemet csak egyszer kell megmozgatnunk.

## Sorbarendezések

### Buborékrendezés

Adott egy számsorunk, amit rendezni szeretnénk. Buborékrendezés során fogjuk az első elemet és összehasonlítjuk a másodikkal, ha cserélni kell őket, megcseréljük, majd lépünk a második elemre és így folytatjuk, amíg a listánk végére nem érünk. Ha végigértünk, újra megismételjük ezt az elejéről. Ezt annyiszor hajtjuk végre, ahány elemünk van.
Ha egyszer végiggyalogol az algoritmus a tömbön és már nem kell cserét végrehajtania, akkor a tömbünk rendezve van.
Ha *n* darab adatot szeretnénk rendezni, akkor maximum $n^2$ összehasonlításra lesz szükségünk.

### Rendezés közvetlen kiválasztással

Hasonlít a buborékrendezésre. Az első elemet hasonlítjuk össze az utána lévőkkel, ha kell cseréljük. Így mire a lista végére érünk a legnagyobb/legkisebb elem lesz a legelején (attól függően, hogy csökkenő vagy növekvő sorrendbe szeretnénk rendezni).
Ezután lépünk a második elemre és ezt hasonlítgatjuk össze az utána lévőkkel. Ezt a metódust követjük, amíg az utolsó elemig nem érünk.
Lépésszáma:

$$\frac{n \cdot (n-1)}{2}$$

### Rendezés iterált beszúrással

Adott a rendezendő adatsorunk, húzunk egy vonalat az első elem után, a vonal utáni első elemet pedig fogjuk és beszúrjuk a helyére a vonal előtti részben. Ezután a vonalat a második elem mögé húzzuk, fogjuk a harmadik elemet és megnézzük, hol lenne a helye a vonal előtti

részben majd oda beszúrjuk. A vonalat ezután megint eggyel hátrébb húzzuk, és így csináljuk, amíg a sor végére nem érünk.
Legrosszabb esetben annyi vizsgálatot kell csinálnunk, ahány eleme van a listánknak. Optimalizálása bináris, felezgető kereséssel történik. Ez csak rendezett sor esetén működik. A sor közepén lévő elemmel összehasonlítjuk a keresett elemet, ebből el tudjuk dönteni, hogy az előtte vagy utána lévő elemek közt kell keresni. Ezután megint az adott fél középső elemével hasonlítjuk össze és így tovább. Legrosszabb esetben *log(n)* db összehasonlítást kell elvégezni.

## Legnagyobb és következő elem megtalálása

Véletlenszerű párokba rendezzük az elemeinket. Minden elempárból kiválasztjuk a nagyobb elemet. Ezeket újból párokba rendezzük, megint kiválasztjuk a nagyobbat. Addig ismételjük, amíg egy elem marad. Ez az elem lesz a legnagyobb.